BrainFuck Interpreter

Faculty of Mathematics and Computer Science

Computability and Complexity Theory

Group: 341

Rachieriu Gheorghe Gabriel

October 15, 2026

Contents

1 What is the BrainFuck Language **2**

2 Implementation Architecture **2**
2.1 The naive approach . . . 2
2.2 The Abstract Syntax Tree . . . 3
2.2.1 What is an Abstract Syntax Tree . . . 3
2.2.2 Why Choosing the AST? . . . 4
2.2.3 Implementation . . . 4
2.2.4 Syntax Checking . . . 4
2.2.5 Runtime Errors Handling . . . 4
2.2.6 Optimizations . . . 5

1 What is the BrainFuck Language

Brainfuck is an esoteric programming language created in 1993 by a Swiss student. Designed to be extremely minimalistic, the language consists of only eight simple commands, a data pointer, and an instruction pointer.

Brainfuck is an example of a so-called **Turing tarpit**: it can be used to write any program, but it is not practical to do so because it provides so little abstraction that the programs get very long or complicated. While Brainfuck is fully Turing-complete, it is not intended for practical use but to challenge and amuse programmers.

Reference: https://en.wikipedia.org/wiki/Brainfuck

Character	Instruction Performed
`>`	Increment the pointer with one cell.
`<`	Decrement the pointer with one cell.
`+`	Increment the cell value at the data pointer.
`-`	Decrement the cell value at the data pointer.
`.`	Print the ASCII equivalent the byte at the data pointer.
`,`	Accept one byte of input, storing its value in the byte at the data pointer.
`[`	If the byte at the data pointer is zero, then instead of moving the instruction pointer forward to the next command, *jump* it *forward* to the command after the *matching* `]` command.
`]`	If the byte at the data pointer is nonzero, then instead of moving the instruction pointer forward to the next command, *jump* it *back* to the command after the *matching* `[` command.

Reference: https://en.wikipedia.org/wiki/Brainfuck

2 Implementation Architecture

For this problem I decided the best approach is to build a script that simulates the tape changes accordingly to the Brainfuck code. The tape is exactly 30.000 cells long and each cells stores an integer of 32 bits(the original problem states each cell is made of an 8bits size).

2.1 The naive approach

The naive approach I implemented iterates through the BrainFuck code and for each instruction changes the tape accordingly. The pointer we're using is basically just an integer that is incremented or decremented accodingly to the X and X instructions.

We handle loops in the following way:

- **Starting a loop (i.e. tape[i] == '['):** If the loop is starting and the tape cell is zero (i.e. tape[pointer] == 0), the loop body is skipped.

  We search forward by incrementing the instruction pointer to find the matching closing bracket], while keeping count of nested loops using a loop rank counter.

  Starting at `loop_rank = 1`, every inner `[` increments the counter and every] `decreases` it; when `loop_rank` reaches `0`, we have found the closing bracket matching.

- **Ending a loop (`bytes[i] == ']'`):** When reaching a closing bracket, if the tape cell is not zero (`tape[pointer] != 0`), the loop must repeat.

  We apply the exact same rank counting logic in reverse: decrementing the instruction pointer, incrementing loop_rank on every `]`, and decrementing it on every `[` until `loop_rank` reaches `0` at the matching opening bracket.

This approach was correct but it provided little to no abstraction, **no window to implement syntax checking, runtime errors checking, tape memory issues checking and optimization**.

2.2 The Abstract Syntax Tree

Looking for a better approach, I came across this page Introduction of Compiler Design and Parse Tree and Syntax Tree. This was looking promising, allowing for better abstraction of my code.

2.2.1 What is an Abstract Syntax Tree

A syntax tree displays the syntactic structure of a program and ignores unnecessary details present in a parse tree. It is a condensed form of the parse tree where operator and keyword nodes are moved to their parent and groups of individual productions are replaced by a single link.

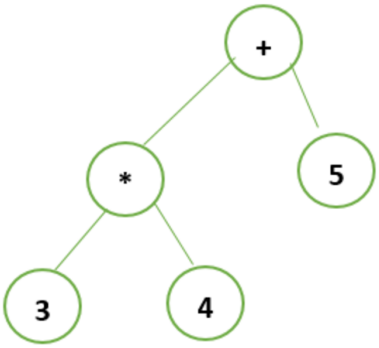
- It is commonly used in compiler design to represent the structure of code during the compilation process.
- It is constructed by parsing the source code which involves breaking it down into components like tokens, variables, and statements.
- The nodes in a syntax tree correspond to various components of the source code, reflecting its grammatical structure.
- Syntax trees are used for tasks such as type checking, optimization, and code generation in compilers.
- They can also be used to represent the structure of other linguistic or logical structures such as natural language sentences or logical expressions.

Let's consider the expression given below:

```
3*4+5
```

Here is the Syntax tree for the expression, `3*4+5`:

- The root node is the `+` operator as addition is the outermost operation in the expression.
- The left child of `+` is the `*` operator which handles the multiplication of `3` and `4`.
- The left and right child of `*` is `3` and `4` respectively while the right child of `+` is `5`.



Reference: https://www.geeksforgeeks.org/compiler-design/parse-tree-and-syntax-tree/

2.2.2 Why Choosing the AST?

Even though the naive approach was functional, with this project I tried to get a better understanding of how programming languages work. It came natural after the first implementation to ask the the next questions:

- **How can I syntax check and syntax highlight the code?**
- **How can I spot errors that might occur at runtime and fix them and parsing stage?**
- **How can I optimize the code to reduce the number of iterations?**

The AST implementation provided me the level of abstraction and control I needed to solve or at least approach these problems.

2.2.3 Implementation

For this implementation we split the logic into two parts: **parsing** and **execution**.

First, we define an `Instruction` enum that represents all Brainfuck allowed instructions: `Inc(u32)`, `Dec(u32)`, `MoveLeft(u32)`, `MoveRight(u32)`, `PrintToAscii`, and `Loop(Vec<Instruction>)`.

Instead of jumping back and forth through string at runtime, our `parser()` function parses the source code once and builds an AST out of these instructions.

To handle nested loops , the `parser()` uses a stack of instruction (`Vec<Vec<Instruction>>`). When the tape is an opening bracket `[`, we push an empty vector in the stack to start collecting instruction for the next the loop.

When we hit the matching `]`, we pop that finished vector off the stack, create an `Instruction::Loop`, and append it to its parent.

Once the tree is built, the `interpret()` function initializes our memory tape of 30,000 cells set to 0, the data pointer, calls `parser()` to get the AST, and passes everything to the recursive `execute()`function.

2.2.4 Syntax Checking

Since we parse the code into a tree before execution we can leverage this arborescent structure to catch bracket mistakes before any instruction touches the tape:

- **Unexpected closing brackets (`]`):** When the parser encounters a `]`, it checks if the stack has at least two levels: the main program level plus the current open loop. If `stack.len() < 2`, it means there was no matching `[` before it, so the parser immediately stops and returns an unmatched bracket syntax error.
- **Unclosed opening brackets (`[`):** After reading the entire source string, the stack should only have one vector left-the main program's AST. If `stack.len() > 1`, it means one or more loops were opened with `[` but never closed, and the parser returns a syntax error.

2.2.5 Runtime Errors Handling

Besides dard syntax errors, our interpreter also looks out for potential bad logic:

- **Infinite loop warnings:** During parsing we inspect the body of a loop. If the loop consists of a single increment instruction, specifically the `[+]`, we print a yellow warning letting the user know the loop at that byte index lacks an ending condition.
- **Boundary and arithmetic safety:** At runtime, moving the pointer left past index `0` or right past index `29,999`, or decrementing a cell below `0`, will trigger Rust's standard bounds and underflow checks rather than silently corrupting memory like in C.

2.2.6 Optimizations

Brainfuck code is full of repetitive characters (like `+++++++` or `<<<<`) is a waste.

I tried to implement these optimizations at parse time and runtime to speed things up:

- **Run-Length Encoding - Parse Time:** Instead of creating five separate `Inc(1)` nodes for `+++++`, our parser checks the last instruction on the stack before pushing a new one. If the new character matches the previous operation (`+`, `-`, `<`, or `>`), we simply increment the counter inside the existing instruction (turning it into `Inc(5)`). This shrinks the AST and allows `execute()` to add, subtract, or move the pointer in a single step.
- **Zeroing Idiom Recognition - Runtime** In Brainfuck, the pattern `[-]` is the standard idiom for resetting the current memory cell to zero. Normally, if a cell holds the value `250`, the interpreter would have to run the loop 250 times, subtracting `1` each time. In `execute()`, before running a loop, we check if its body contains just a single `Instruction::Dec(1)`. If it does, we skip the loop mechanics and directly set `tape[*pointer] = 0` in $O(1)$ time.